C0 Start here: foundations for Fabric

Data platform vocabulary, Power BI semantic models, and the Spark and SQL engines side by side.

C0-L1 Data platform vocabulary, files, tables, lakes and warehouses

Keep data once, in a typed table format such as Delta, and let each engine read it, instead of exporting copies to untyped files.

CSV, Parquet and Delta	CSV: text, no types. Parquet: columnar, typed, compressed. Delta: Parquet files plus a transaction log with versions.
Lake, warehouse, lakehouse	Lake: files, structure on read. Warehouse: typed tables for SQL. Lakehouse: open tables in a lake, read by Spark and SQL.
ELT and medallion stages	Load raw first, transform inside the platform: bronze (as received), silver (clean), gold (aggregated).
Fabric's names for these	OneLake is the one lake (GA). Lakehouse tables are Delta (GA). The SQL analytics endpoint queries them with T-SQL (GA).

C0-L2 Power BI basics, reports, semantic models and measures

Put ratios in measures that divide totals, and let filter context decide what each total covers.

```
Weighted reach % = DIVIDE ( SUM ( fact[reach] ), SUM ( fact[base] ) )
```

Data, semantic model, report	Data sits in source tables. The model describes tables, relationships and measures. Reports hold visuals that query the model.
Calculated column against measure	A column is worked out per row and stored. A measure is worked out when a visual asks, in that visual's filter context.
Ratios in Power BI	A measure dividing totals: DIVIDE (SUM (reach), SUM (base)). Never average per-row percentages.
Import, DirectQuery, Direct Lake	Import copies and refreshes. DirectQuery queries the source each time. Direct Lake (GA) reads OneLake Delta tables directly.

C0-L3 Two engines, one table, Spark notebooks and the SQL analytics endpoint

One Delta table, two engines: write and transform with Spark, query with T-SQL on the SQL analytics endpoint.

```python
from pyspark.sql import functions as F

df = spark.read.table("respondents")
totals = df.filter(F.col("final_weight") >
  0).groupBy("region").agg(F.sum("final_weight").alias("total"))
totals.show()
```

```sql
SELECT region, SUM(final_weight) AS total
FROM dbo.respondents
WHERE final_weight > 0
GROUP BY region;
```

Two engines over one lakehouse table	Spark in notebooks reads and writes. The SQL analytics endpoint queries with T-SQL and is read-only for table data.

Append rows to a lakehouse table from Spark	df.write.mode("append").saveAsTable("respondents"). mode("overwrite") replaces the rows.
Spark transformations against actions	filter, groupBy, agg, select describe a new DataFrame. show, count, collect run the plan.
Where to put T-SQL writes	In a Warehouse. The lakehouse's SQL analytics endpoint does not write table data.